BOOX STORE

From Cart to Checkout

What the code does, how payment is confirmed, and what you need to run a safe test.

YOUR BROWSER ? OUR SERVER ? STRIPE

The shopper chooses books. The server checks real prices and creates an order. Stripe securely collects test payment.

The choices we are using

Stripe-hosted Checkout ? signed-in customers ? USD ? demo only, no shipping or tax

You do not have a Stripe account yet, so the code is ready for test-mode setup but a real Stripe payment could not be run yet.

1. The checkout trip

Step	What happens
1. Click	The shopper presses ?Proceed to secure checkout.? The browser sends only book IDs and quantities?not prices.
2. Sign in	The server checks the Better Auth session. Orders belong to signed-in users in the current database design.
3. Check	The server validates the cart, looks up each book in PostgreSQL, and calculates the amount in cents using database prices.
4. Save	The server saves a pending order and its line items, including the price at the time of this order.
5. Pay	The server creates a Stripe Checkout Session and sends the browser to Stripe?s secure hosted checkout page.
6. Confirm	Stripe sends a signed webhook to the server. The server verifies it and only then marks the order paid.
7. Return	The store checks the signed-in customer?s order status. It clears the browser cart only after the database says paid.

Why the server checks prices

A browser is like a note a shopper can edit. Someone could change numbers sent from their own browser. The server therefore accepts IDs and quantities, then looks up prices from its trusted catalogue. Stripe receives those server-calculated prices.

Why a webhook matters

Returning to a ?thank you? page is not proof of payment. The customer might close the page or visit a URL without paying. Stripe?s signed webhook is the server-to-server proof. The code checks the signature, order ID, Checkout Session ID, currency, and total before setting **status = paid**.

This demo does not add shipping or tax. It charges the book subtotal in USD.

2. Where the code lives

File	Purpose
frontend/src/components/CartSection.jsx	Sends the cart to the server, redirects to Stripe, and checks payment status after returning.
server/src/checkout-routes.js	Creates Checkout Sessions, looks up a customer?s own order, and receives/verifies Stripe webhooks.
server/src/cart-validation.js	Rejects empty, duplicate, malformed, or unreasonable cart items before creating an order.
server/src/db/schema.js	Describes orders and order_items in Drizzle. It stores order status, cent totals, Stripe Session ID, and purchased quantities/prices.
server/src/db/migrations/0001_create_orders.sql	Creates the order tables for a database that does not have them yet. It uses IF NOT EXISTS.
server/src/index.js	Mounts the checkout routes on the Express server.
server/.env.example	Shows the names of Stripe settings. Real keys belong in the ignored server/.env file.
server/src/cart-validation.test.js	Tests valid carts and rejects empty, duplicate, too-large, or invalid quantities.

Order states

pending means the order exists but payment is not confirmed. **paid** is set only by a valid Stripe payment webhook. **expired** means Checkout timed out. **failed** means an asynchronous payment failed.

Webhook duplicates are expected

Payment services may deliver the same event more than once. The server updates only pending orders, so a repeated event does not create another payment or change an already-paid order.

The configured database already contains both order tables and the expected columns. The SQL migration is supplied for another or newly created database; do not run it against a different database without checking first.

3. What you need to set up

You said you do not have a Stripe account yet. That is okay: you can create one and test without using real money. Never send secret keys in chat, screenshots, source code, or a committed file.

A. Make a Stripe test account

- Create/sign in to a Stripe account and switch to test mode in the Stripe Dashboard.
- Copy the **test secret key** (starts with sk_test_). A publishable key is not needed for this hosted Checkout flow.
- In **server/.env**, set **STRIPE_SECRET_KEY=sk_test_...**. This file is ignored by Git. Never put this key in frontend code.

B. Forward webhooks while developing locally

- Install Stripe CLI using Stripe?s official instructions, then authenticate it with **stripe login**.

```
stripe listen --forward-to localhost:3000/api/stripe/webhook
```

- The CLI prints a webhook signing secret starting with **whsec**_. Put it in server/.env as **STRIPE_WEBHOOK_SECRET=whsec_...**.
- Keep the Stripe CLI listener running while testing. Restart the server after changing environment values.

C. Start the app

```
Terminal 1:  cd server   then   npm run dev
Terminal 2:  cd frontend then   npm run dev
```

The server needs its existing DATABASE_URL, BETTER_AUTH_SECRET, BETTER_AUTH_URL, and CLIENT_URL values as well as the two Stripe settings.

The configured database was checked read-only and already has the orders and order_items tables. For a different database, first confirm they are absent, then apply server/src/db/migrations/0001_create_orders.sql in that database.

4. Test it safely

- Sign in to the store. The current checkout requires an account.
- Add one or more books, open the cart, and select **Proceed to secure checkout**.
- Stripe Checkout should show the book titles and server-calculated USD prices.
- In Stripe test mode, use card number **4242 4242 4242 4242**, any future expiry date, and any three-digit CVC. Use a test name/address if asked.
- Wait for Stripe CLI to forward the webhook. The store should report payment confirmed and empty the cart only after that confirmation.
- Check the Stripe test Dashboard and the database order row. It should have status paid, total_cents, stripe_session_id, and paid_at set.

If something goes wrong

What you see	First thing to check
Checkout is not configured	Confirm STRIPE_SECRET_KEY is present in server/.env and restart server.
Payment stays ?being confirmed?	Keep stripe listen running; confirm STRIPE_WEBHOOK_SECRET matches the current CLI listener secret.
Please sign in	Sign in to the store before checkout; return to the same account after payment.
Could not create order	Check server logs and confirm the database has the tables described by the SQL migration.

Going live is a separate step

Before accepting real payments, you must choose your real shipping, tax, refund, and fulfilment policies. This version deliberately has no shipping or tax. Then configure Stripe live mode and a production webhook endpoint at **https://your-store-domain/api/stripe/webhook**, subscribe it to checkout.session.completed, checkout.session.async_payment_succeeded, checkout.session.async_payment_failed, and checkout.session.expired, and store the live secrets in the hosting provider?s secret settings?not in Git.

Validation completed for this change: frontend lint and production build pass; server cart-validation tests pass; the server checkout routes load. A Stripe payment itself still needs your test account keys and a running webhook listener.